

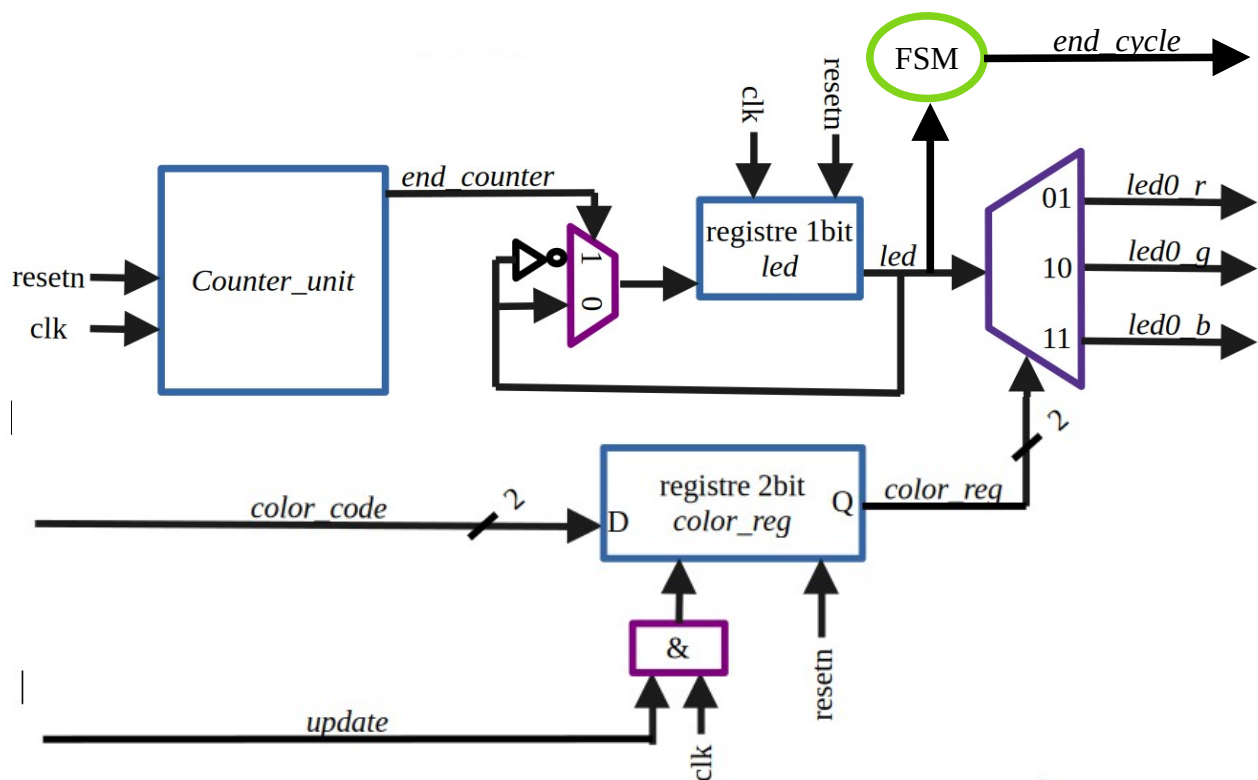
TP5

Domaines d'horloge

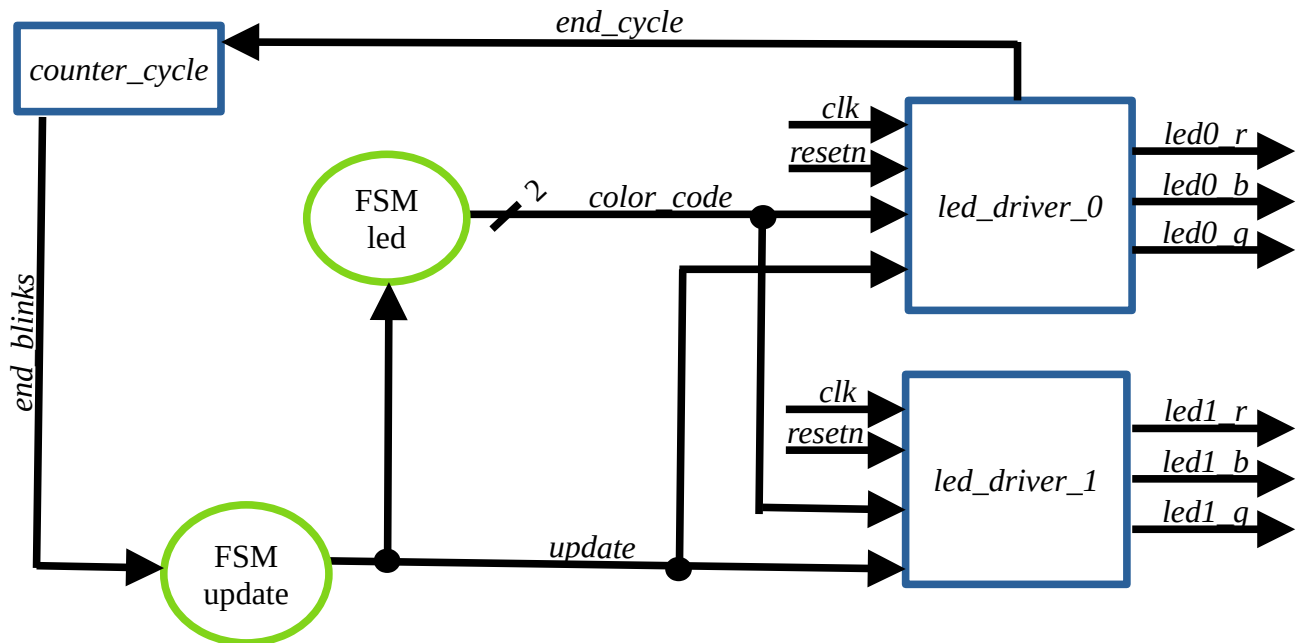
L'objectif du TP est de piloter deux leds avec deux horloges différentes.

1)

Tout d'abord, on veut piloter deux leds avec une seule horloge.
Deux modules *led_driver* sont utilisés. On rappelle sa structure ci-dessous.

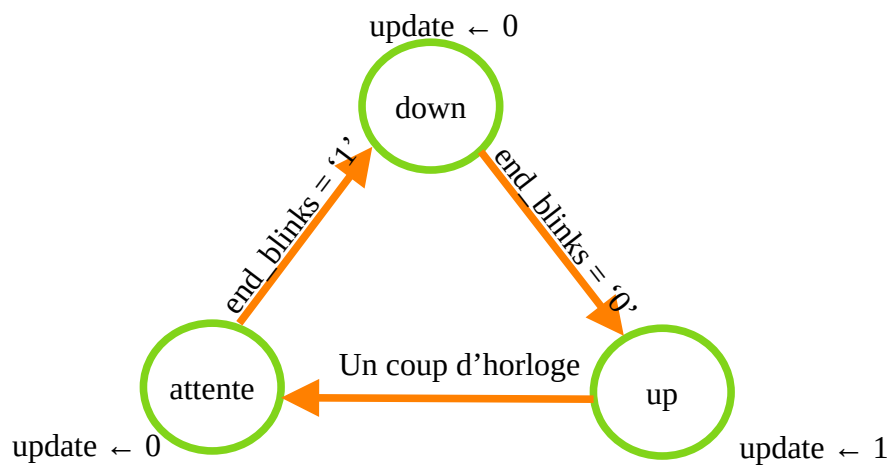


La structure de *top_led_driver* utilise deux machines d'état : une pour le signal *update* l'autre pour le signal *color_code*. Un compteur sert à la FSM de *color_code* pour compter 10 clignotement des leds.

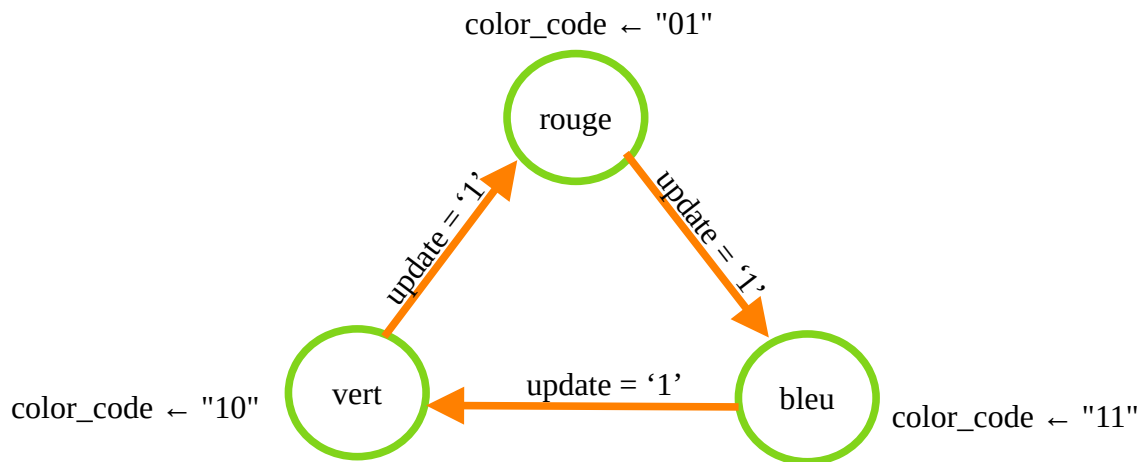


Le compteur *counter_cycle* compte jusqu'à 10. Il a donc une taille de 4bits. Il prends en entrée le signal de fin de cycle de clignotement provenant des *led_driver*. Ici, les deux modules fonctionnent avec la même horloge donc les clignotements se terminent en même temps. Les deux signaux *end_cycle* sont en fait identiques. On peut se permettre ici d'en utiliser qu'un seul des deux.

La machine d'état pilotant *update* est identique à celle construite dans le TP4. *update* est à 1 pendant un seul coup d'horloge lorsque *end_blinks* passe de 1 à 0. Cela permet de faire la mise à jour après le 10^{ème} clignotement et non pas au début.



La machine d'état qui pilote *color_code*, a trois états : rouge, bleu et vert. À chaque réception du signal *update*, la machine change d'état.

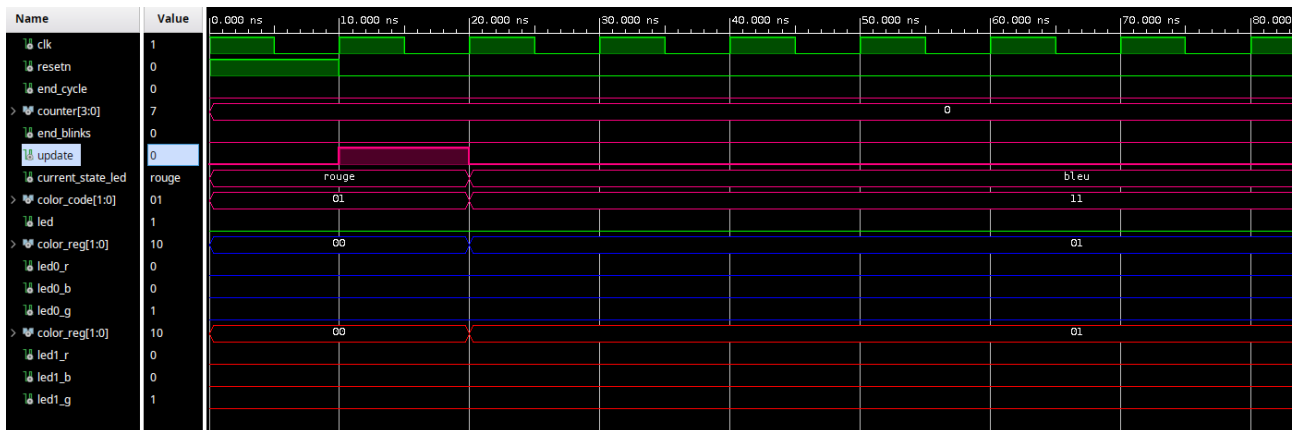


3)

Les fichiers sources et le testbench sont nommés *top_led_driver_Q3* et *tb_led_driver_Q3*.

Pour la simulation, le threshold est passé à 20 cycles.

Sur les chronogrammes, en rose les signaux de *top_led_driver*, en bleu le module *led_driver_0* et en rouge le module *led_driver_1*.

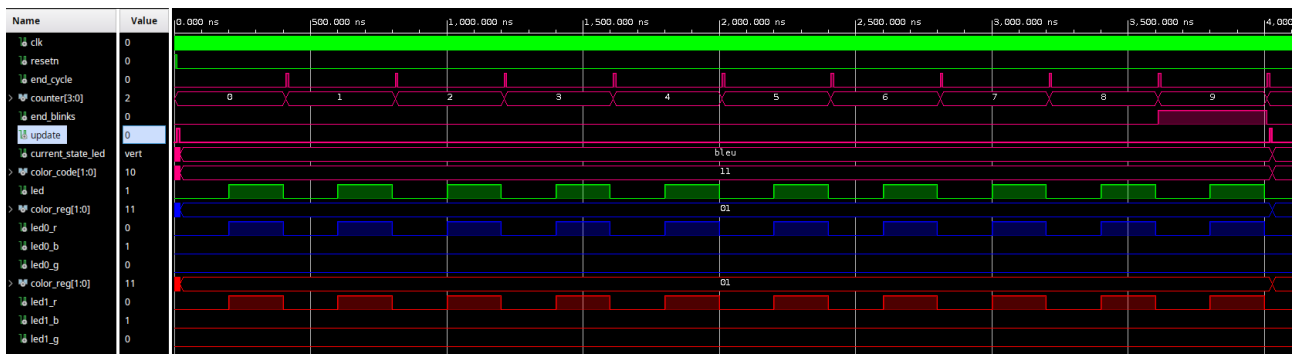


Juste après le reset, la FSM des leds est à l'état rouge et *color_code* est à 01₂ (Rouge)

Le compteur de clignotement est à 0.

Les leds sont éteintes et les deux *color_reg* sont à 00₂ (aucune couleur).

Le signal *update* est à 1 tout de suite après le reset. Ce signal fait changer l'état de la FSM à bleu et les *color_reg* prennent la valeur de *color_code*.



À chaque fin de clignotement (*end_cycle*), le compteur de clignotement s'incrément.

Les leds rouges clignotent.

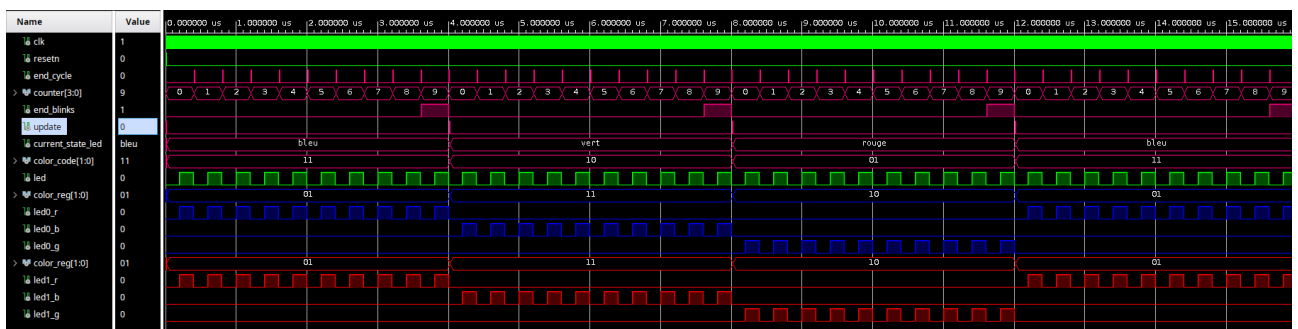
Le compteur compte de 0 à 9 soit 10 clignotements.

Là, le signal *end_blinks* est levé ce qui déclenche le signal *update*.



Les leds bleues se mettent à clignoter à leur tour.

La machine d'état passe à vert.

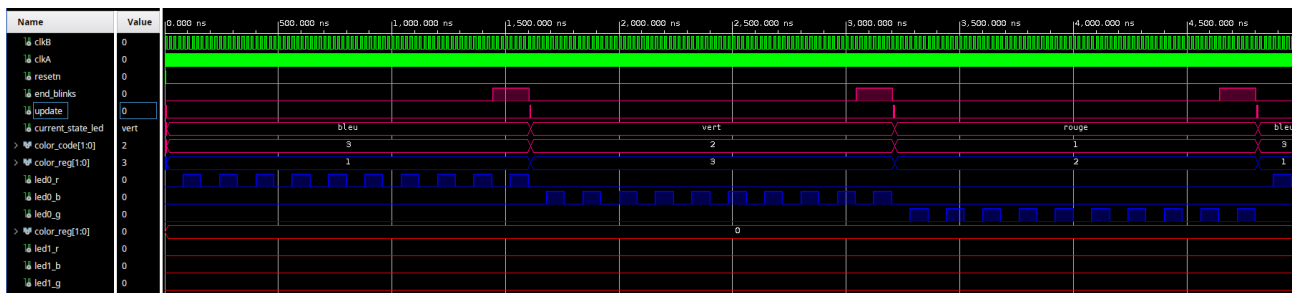


Les leds respectent bien 10 clignotement de chaque couleurs dans l'ordre imposé.

Après une séquence, la led rouge redémarre.

6)

Le module *led_driver_0* fonctionne avec *clkA* à 250MHz et le module *led_driver_1* avec *clkB* à 50MHz



La led 0 clignote 10 fois avant de passer à la couleur suivante. Le fonctionnement du module *led_driver_0* est normal. La led 1 quant à elle ne s'allume pas du tout.



Le signal *update* ne dure qu'une période de l'horloge A.

L'horloge B, plus lente, ne parvient pas à détecter *update*. Le front montant de *clkB* a peu de chance de tomber pendant que *update* est à 1.

Le signal n'est donc pas reçu par le module *led_driver_1*, le registre *color_reg* n'est pas mis à jour et donc la led 1 ne s'allume pas.

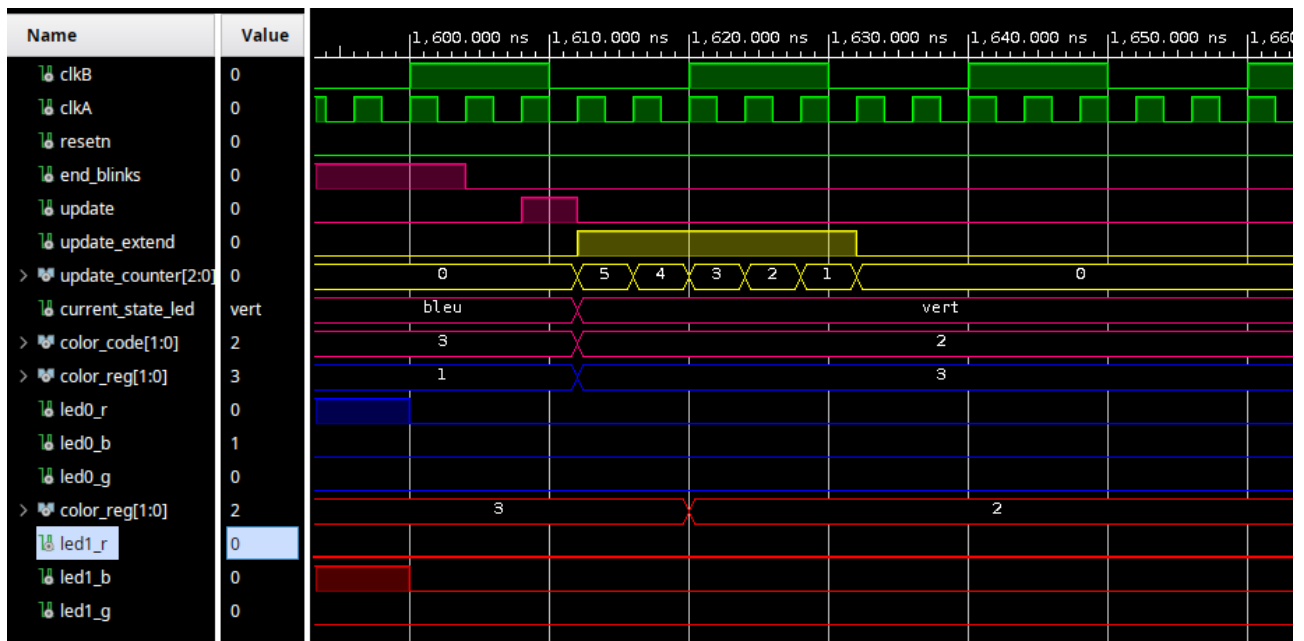
7)

Une solution possible est d'étendre le signal *update* avant de l'envoyer à *led_drive_1*.

Le signal étendu va durer 5 cycles de *clkA* soit 1 cycle de *clkB*. Cela nécessite un compteur de 3 bits pour aller jusqu'à 5.

8)

En jaune les deux signaux propres à l'extension du signal *update*.



Le signal *update_extend* est 5 fois plus long que *update*. Le compteur *update_counter* sert à étendre le signal.

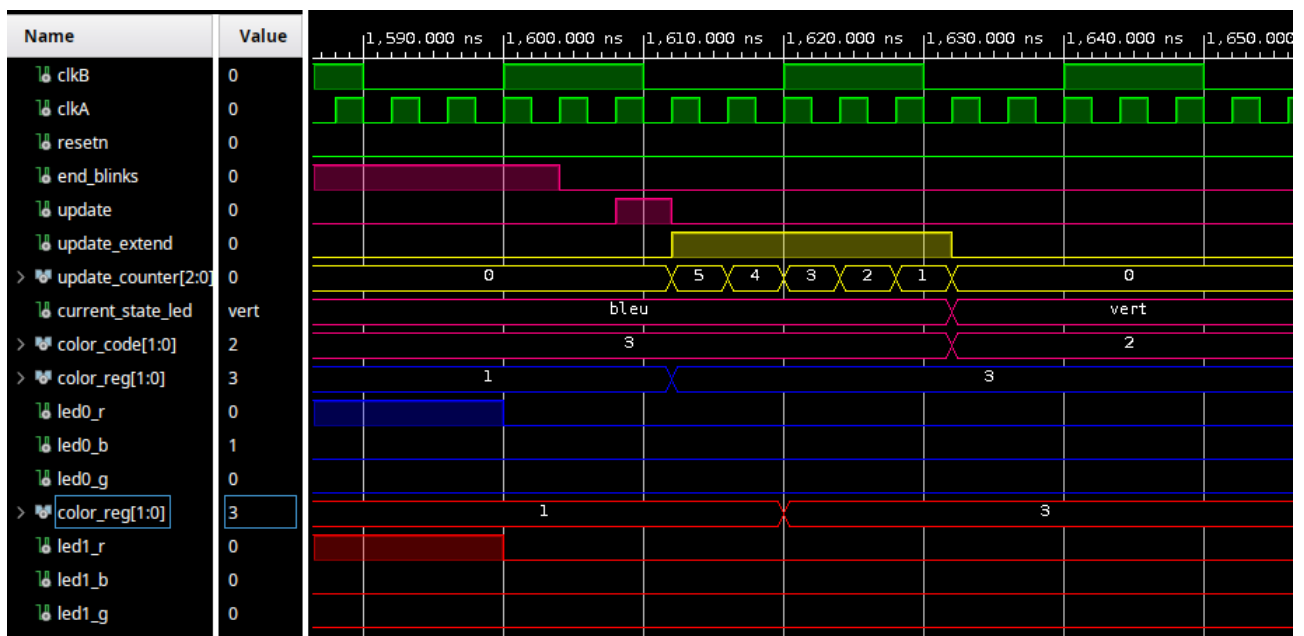
update_extend est bien reçu par le module de la led 1 et met à jour son registre *color_reg*.

Néanmoins, on remarque que la valeur enregistrée dans *color_reg* est celle de la couleur suivante.

Le signal *update_extend* est reçu trop tard ou la valeur de *color_code* change trop tôt.

Pour corriger ce problème, j'ai choisi de changer les conditions de changement d'état de la FSM des leds.

La machine change d'état lorsque *update_counter* est à 1. Cela garanti un délais suffisant pour que la bonne commande de couleur soit récupérée par *led_driver_1*.

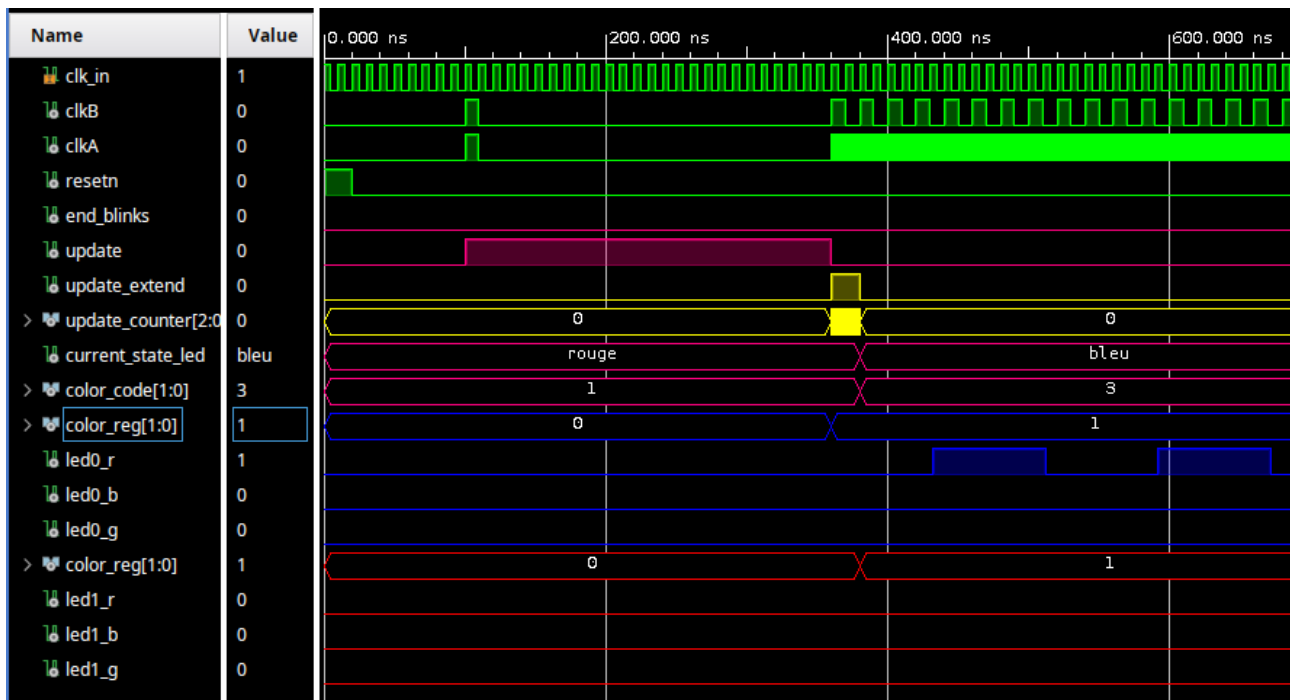


Ici, les leds passent du rouge vert le bleu. L'état courant de la FSM reste plus longtemps à « bleu » pour laisser le temps au module de la led 1 de récupérer la commande.
L'état passe à « vert » quand *update_counter* est à 1, c'est-à-dire lorsque *update_extend* revient à 0.
On voit que le registre *color_reg* de la led 1 est correctement mis à jour.

9)

L'IP de PLL utilisé s'appelle Clocking Wizard. Il est rédigé en Verilog mais s'intègre de la même manière à notre architecture.

À la simulation, on vérifie que les horloges sont bien générées.



Les horloges A et B sont générées avec les fréquences de 250MHz et 50MHz comme attendu.
On remarque un délai de création des signaux d'horloges de 360ns.
Le comportement des signaux pendant ce délai semble étrange mais n'affecte pas le fonctionnement du système.

10)

La synthèse permet d'obtenir le schematic suivant.

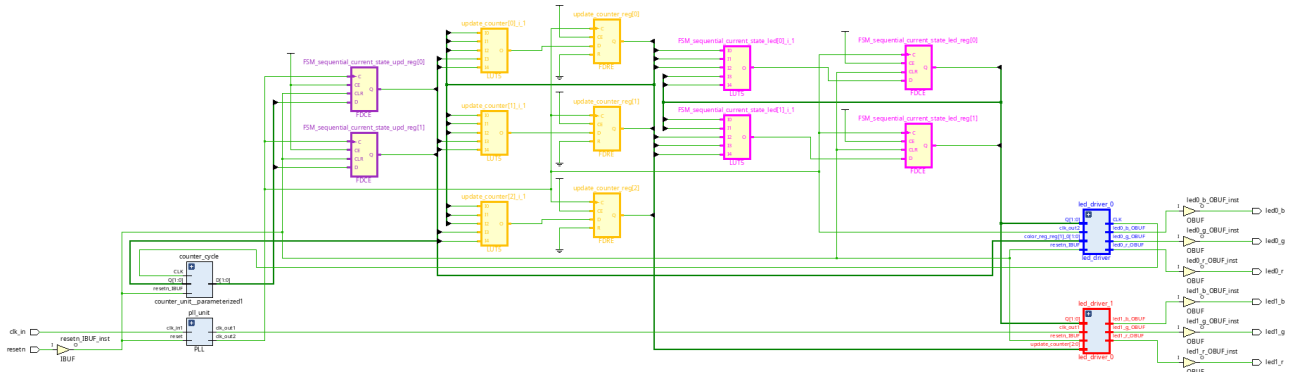
En bas à gauche, la PLL et le compteur de cycles.

En violet, la FSM du signal *update*.

En orange, le compteur d'extension de *update*.

En rose, la FSM pour le signal *color_code*.

À droite, les deux modules *led_driver_0* et *led_driver_1*.



11)

Le rapport de synthèse montre l'utilisation matérielle suivante :

7. Primitives

Ref Name	Used	Functional Category
FDCE	73	Flop & Latch
LUT2	60	LUT
CARRY4	14	CarryLogic
LUT3	10	LUT
LUT6	9	LUT
LUT4	9	LUT
LUT5	7	LUT
OBUF	6	IO
LUT1	1	LUT
IBUF	1	IO

Tous les éléments logiques sont réparti comme suit :

	Registres FDCE	LUT	CARRY
<i>counter_cycle</i>	4	4	
<i>update_counter</i>	3	3	
<i>FSM update</i>	2	2	
<i>FSM color_code</i>	2	2	
<i>led_driver_0</i>	5	8	
<i>led_driver_1</i>	3	5	
<i>led_driver/counter (x2)</i>	54 (27x2)	72 (36x2)	14 (7x2)
Total	73	96	14

Dans le module *led_driver_1*, le signal *end_cycle* n'est pas lu dans le top. Vivado a simplifié ce signal et a supprimé la machine d'état associée. Cela libère donc 2 FDCE et 2 LUT pour la FSM et 1 LUT pour la sortie du module.

L'IP de la PLL est bien synthétisée.

8. Black Boxes	
Ref Name	Used
PLL	1

12)

Lors de l'implémentation, des erreurs de timing apparaissent. Celles-ci concernent l'horloge A à 250MHz de la PLL à l'ILA et au sein de compteur de *led_driver_0*.

Nous en avons conclu que la carte ne support pas une fréquence aussi élevée que 250MHz.

La décision a été de baisser la fréquence de *clkA* à 200MHz.

Cela résout le problème de slack dans le compteur de *led_driver_0*. Mais l'ILA montre toujours des erreurs de timing.

Setup		Hold	
Worst Negative Slack (WNS):	0,470 ns	Worst Hold Slack (WHS):	0,123 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	69	Total Number of Endpoints:	69

Les TNS et THS sont maintenant à 0ns. Le chemin critique se situe dans le même compteur du module *led_driver_0*. Son slack est de 0,470ns.

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 41	0.470	10	29	led_driver_0/co...unter_reg[13]/C	led_driver_0/co...unter_reg[25]/D	4.496	2.264	2.232	5.0	clk_out2_PLL	clk_out2_PLL
Path 42	0.565	10	29	led_driver_0/co...unter_reg[13]/C	led_driver_0/co...unter_reg[26]/D	4.401	2.169	2.232	5.0	clk_out2_PLL	clk_out2_PLL
Path 43	0.581	10	29	led_driver_0/co...unter_reg[13]/C	led_driver_0/co...unter_reg[24]/D	4.385	2.153	2.232	5.0	clk_out2_PLL	clk_out2_PLL
Path 44	0.585	9	29	led_driver_0/co...unter_reg[13]/C	led_driver_0/co...unter_reg[21]/D	4.382	2.150	2.232	5.0	clk_out2_PLL	clk_out2_PLL

La *clk_out2* est générée par la PLL à 200MHz et est connectée à *clkA*.

On remarque que les chemins les plus long proviennent tous du même registre du compteur ; ici le registre *counter_reg[13]*.

La fréquence plus basse de *clkA* ne règle pas les problèmes de timing avec l'ILA.

Quand l'ILA est configurée pour échantillonner les signaux d'horloges, les délais de Hold ne sont pas respectés.

Setup		Hold	
Worst Negative Slack (WNS):	0,287 ns	Worst Hold Slack (WHS):	-0,548 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	-1,288 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	3
Total Number of Endpoints:	5798	Total Number of Endpoints:	5782

Le TNS est nul mais le THS est négatif. Les chemins problématiques sont tous entre la PLL et l'ILA.

13)

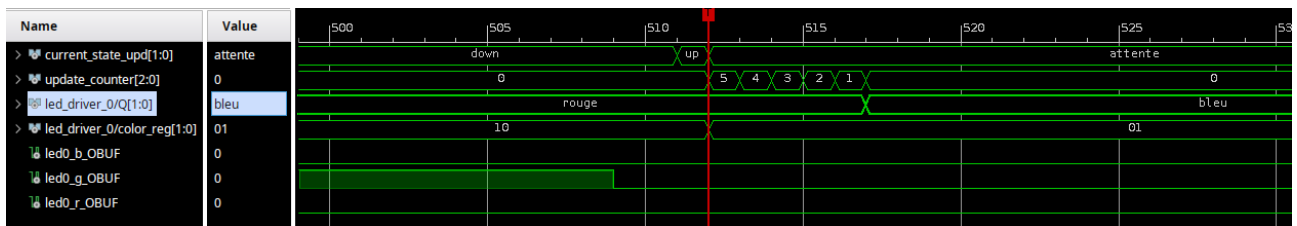
La génération du bitstream et la programmation de la carte se déroulent correctement.

Les leds clignotent en suivant la séquence rouge, bleu, vert puis à nouveau rouge.

La led 0 clignote 10 fois avant de changer de couleur. La led 1 clignote plus lentement et suit la même séquence couleur.

Il n'est pas possible d'observer les horloges dans l'ILA car cela créer des erreurs de timing. Les autres signaux sont observables.

On observe ici un changement de couleur de vert vers rouge.



La led 0 clignote en vert et l'état de la machine de *color_code* est à rouge.

La FSM du signal *update* est à up. Cela provoque le changement d'état de la FSM de *color_code*.

Le compteur *update_counteur* est déclenché. Cela montre que le signal *update* est étendu sur 5 cycles de *clkA* pour que *led_driver_1* puisse le capter.

Les registres des module de led prennent la valeur de rouge (01₂).